

Registros de E/S del PIC18

ANSEL · TRIS · LAT · PORT — qué hace cada uno y cuándo se usa

No son cuatro formas de hacer lo mismo: son cuatro registros que controlan **etapas distintas** del mismo pin. Un pin físico atraviesa varias capas de configuración y cada registro manda en una de ellas. Piensa en cada pin como una puerta con varios interruptores en serie.

ANSEL — ¿analógico o digital?

Decide la naturaleza del pin. Muchos pines pueden funcionar como **entrada analógica** (medir un voltaje con el ADC) o como **pin digital** (que solo entiende de 0 y 1). Un bit a 1 = analógico; a 0 = digital.

Consecuencia crítica: **si un pin se queda en analógico, su lectura digital siempre da 0**. Por eso en el código va `clr ANSELB` antes de leer el botón: sin eso, `PORTB, 0` devolvería siempre 0 y el pulsador nunca se detectaría.

TRIS — ¿entrada o salida?

Una vez el pin es digital, TRIS decide su **dirección**: si escucha (entrada) o si manda (salida). Regla: **1 = entrada, 0 = salida** (el 1 recuerda a la “I” de *Input*). Es lo que hace que RB0 sea entrada (botón) y RC6 salida (LED).

LAT — el valor que SACO por una salida

LAT (de *latch*, “cerrojo”) es donde **escribes** lo que quieres que salga por un pin configurado como salida. Bit a 1 → el pin sube a nivel alto; a 0 → baja. Es el registro que enciende y apaga el LED.

PORT — el valor que ENTRA de verdad por el pin

PORT es lo que **lees** para saber el estado eléctrico real del pin en ese instante. Es lo que usas para el botón: `bt fsc PORTB, 0` mira si por RB0 entra un 1 o un 0.

La regla de oro

LAT y PORT parecen lo mismo porque ambos “son el puerto”, pero se usan en sentidos opuestos:

Si quieres...	Usa
Poner un pin analógico o digital	ANSEL
Definir si un pin es entrada o salida	TRIS
Escribir una salida (encender / apagar)	LAT
Leer una entrada (estado real del pin)	PORT

En una frase: **las salidas se escriben en LAT y las entradas se leen en PORT**. En el programa se cumple al pie de la letra — el LED (salida) se toca siempre con `LATC` y el botón (entrada) se lee siempre con `PORTB`.

¿Por qué se escribe en LAT y no en PORT?

Si escribes salidas usando PORT con instrucciones bit a bit (`bsf` / `bcf`), el PIC primero *lee* el pin real, modifica el bit y lo vuelve a escribir. Si el pin aún no ha llegado al voltaje que pusiste (por una carga, un condensador...), esa lectura sale mal y puedes corromper otros bits del puerto. Escribiendo en LAT lees el cerrojo, no el pin, y el problema desaparece. Es el famoso *read-modify-write*; para empezar, basta con seguir la regla de oro.